

Assignment 1 :

ChatGPT ▾

Log in Sign up

[Context] : As a Software test lead generate a test strategy document on Ecommerce payment module Ask me Exact 5 question step by step to create the test strategy document . Test document must be in Professional tone and as per industry standard step by step implementation.

Perfect! To create a **professional, industry-standard Test Strategy Document** for an **E-commerce Payment Module**, I'll ask **5 precise questions**, one at a time, to capture all necessary details. Let's start with the first:

Question 1:
What is the **scope of the payment module** to be tested? For example, are we covering only the front-end payment workflows, backend transaction processing, third-party payment gateway integrations, refunds, and/or wallet features?

This will define the boundaries of the test strategy.

 

third-party payment gateway integrations, refunds, and/or wallet features

↓

Question 2:
Can you specify the **types of payment methods and gateways** that need testing? For example: credit/debit cards, net banking, UPI, digital wallets (Paytm, Google Pay, etc.), EMI options, or any region-specific payment providers. This will help define test coverage and scenarios.

 

Paytm, Google Pay, etc.

↓

Question 3:

What are the **non-functional requirements** we should consider for this payment module? For example: performance (transaction speed), security (PCI DSS compliance, encryption), scalability, and availability. This will guide load, security, and reliability testing in the strategy.



no security , no non functional

next-q

Question 4:

What is the **target environment and platforms** for testing? For example: web browsers (Chrome, Firefox, Safari), mobile apps (iOS, Android), or both, including any specific OS versions. This ensures the strategy covers compatibility.



chrome

Next Q-

Question 5:

What is the **test data and transaction volume** expectation? For example: multiple user accounts, different wallet balances, successful and failed payments, partial refunds, etc. This will help define test scenarios and data setup in the strategy.



multiple user accounts, different wallet balances

Final output:

Test Strategy Document – E-commerce Payment Module

Prepared by: [Software Test Lead]

Date: [Insert Date]

Version: 1.0

1. Introduction

This document outlines the **test strategy** for the **E-commerce Payment Module**, focusing on **functional testing** of third-party payment gateway integrations, refunds, and wallet features. The objective is to ensure the module works as intended and handles different

2. Scope of Testing

The following areas are **in-scope**:

- Integration with third-party payment gateways (e.g., Paytm, Google Pay).
- Refund processing for completed transactions.
- Wallet functionalities, including balance handling and partial payments.

Out of scope:

- Security testing (PCI DSS, encryption).
- Non-functional testing (performance, load, scalability).
- Cross-browser or cross-platform testing beyond Chrome.



3. Test Approach

3. Test Approach

The testing approach will be primarily **manual functional testing**, with the following focus areas:

3.1 Payment Gateway Integration Testing

- Verify successful and failed transactions with Paytm and Google Pay.
- Validate correct redirection and callbacks from payment gateways.
- Check error handling for gateway failures, cancellations, or timeouts.

3.2 Refund Testing

- Test full and partial refunds.
- Verify wallet balance updates after refunds.



Assignment 2: Design the automation framework structure using a chain-of-thought approach

Output: Analyze and review the generated results

ChatGPT

Log in Sign up

[Context] : As a Senior SDET Generate a POM playwright javascript automation framework to automate Ecommerce domain application. Ask me exact 5 question one by one to understand my requirement over framework .
Go to next step only once step 1 is clear Follow the Step by step analysis technique . Task - Follow Page object module , use a config file for URL and other dependency

Perfect! Let's approach this step by step. To make sure the Playwright POM framework fits your eCommerce automation needs perfectly, I'll ask you **five questions one by one**. We'll move to the next only once the current step is crystal clear.

Question 1

Step 1: Understanding the Scope of the Application

Question 1:

Which specific functionalities of the eCommerce application do you want to automate? For example:

- User authentication (login/logout)
- Product search and filtering
- Add to cart / checkout
- Order history / payment
- Admin portal functionalities

I need a clear list so I know which pages and modules we need in the Page Object Model.



Next

Step 2: Understanding Test Data & Environment

Question 2:

Do you want the framework to support **multiple environments** (like QA, Staging, Production) and configurable test data, or is it just a single environment with hardcoded test data?

This will help determine how we structure the **config file** and data handling in the POM framework.

next

Question 3:

Which **browsers** do you want to run the tests on (Chromium, Firefox, WebKit), and do you need **headless or headed mode** by default?

This will define the Playwright **browser launch settings** in the framework.



Chromium

next

Step 4: Test Reporting & Logging

Question 4:

What kind of **test reporting and logging** do you need in the framework? For example:

- Console logs only
- HTML / JSON / Allure reports
- Screenshots or video on failure

This will guide how we integrate reporting into the POM framework.



next

Step 5: Framework Features & CI/CD

Question 5:

Do you want the framework to support **CI/CD integration** (like GitHub Actions, Jenkins) and parallel execution, or is it just for local test runs?

This determines how we structure **test runners, scripts, and parallelization** in the POM framework.



Result:

```
ecommerce-automation/
├── tests/                                # Test cases
│   ├── addToCart.test.js
│   └── checkout.test.js
├── pages/                               # Page Object Model classes
│   ├── BasePage.js
│   ├── HomePage.js
│   ├── ProductPage.js
│   ├── CartPage.js
│   └── CheckoutPage.js
├── config/                             # Configuration files
│   └── config.json
├── utils/                              # Helper utilities (logging, data, screenshc
└──
```

```
| utils/                # Helper utilities (logging, data, screensh
|   └─ logger.js
|
| reports/              # HTML / JSON / Allure reports
|
| playwright.config.js  # Playwright test configuration
└─ package.json        ↓
```